

Cloud Djata - Exploring Mental Health Data



1. Table of Contents

- [1. Table of Contents](#)
- [2. Project Planning and Task Distribution](#)
- [3. Presentation Video](#)
- [4. Project Overview](#)
- [5. Data Exploration](#)
 - [5.1. Data Understanding](#)
 - [5.2. Key Observations and Insights](#)
 - [Structurally Missing Data](#)
 - [Noisy Nominal Features](#)
 - [Imbalance Class Distribution](#)
 - [Distribution Visualization](#)
- [6. Data Preprocessing](#)
 - [6.1. Categorical features with only 2 values](#)
 - [6.2. Noisy categorical and nominal features](#)
 - [6.3. Filling Structurally missing values](#)
 - [6.4. Imputing Actual Missing Values](#)
 - [6.5. Processing Duplicated Rows](#)
- [7. Feature Engineering](#)
- [8. Model Development](#)
 - [8.1. Tree-based Ensemble Model](#)
 - [8.2. Stacking Ensemble Model](#)
 - [8.3. MLP \(Multilayer Perceptron\) Model](#)
- [9. Experiments and Results](#)
 - [9.1. Hyperparameter Tuning](#)
 - [9.2. Evaluation Results](#)
 - [Tree-based Ensemble Model](#)
 - [Stacking Ensemble Model](#)
 - [MLP Model](#)
 - [9.3. Test Results](#)
- [10. Conclusion](#)
- [11. Future Work and Improvements](#)
- [12. Screenshot Evidence of All Submissions](#)
- [13. References](#)

2. Project Planning and Task Distribution

Group name: Cloud Djata

Task Distribution:

Student ID	Full Name	Assigned Tasks	Completion Evaluation (%)
22127057	Đỗ Phan Tuấn Đạt	- Perform data exploration and design MLP model, and film presentation videos for said sections - Write final report - Edit and compile presentation videos into one	100
22127064	Phạm Thành Đạt (L)	- Design stacking ensemble model and film presentation video for said section	100
22127449	Mai Đức Vân	- Perform data preprocessing and film presentation video for said section	100
20127011	Lê Tấn Đạt	- Design XGBoost model and film presentation video for said section	95 (late submission)

3. Presentation Video

Youtube link: https://youtu.be/R_zljgPFeol

4. Project Overview

This project develops and evaluates machine learning pipelines in Python programming language to predict a binary depression label from a synthetic survey dataset. It performs extensive data cleaning like, imputing missing values and standardizing categorical ones, along with minor feature engineering. Afterwards, multiple modeling strategies are applied, including gradient-boosted trees, a multilayer perceptron, and a stacked ensemble of tuned

base learners, all with repeated K-fold cross-validation to assess generalization. Models are compared using precision, recall, F1 and accuracy, and final ensembles are trained on the full processed data to produce competition-ready predictions. The workflow tries to emphasize reproducible preprocessing, careful handling of class imbalance, and blended model approaches to maximize predictive performance.

5. Data Exploration

Limited visualization is included in this section for illustration purposes. For full visualization of all features, please refer to the *data_exploration.ipynb* file.

5.1. Data Understanding

The dataset of this competition is synthetic, having been generated by a model which learned from a survey dataset (Sharma, n.d.). Neither the competition nor the original dataset it was based on provided any details on what each feature means. Nevertheless, we can still figure out the meaning of each column based on their names and values. That's said, the features in the dataset goes as follow:

- **id**: ID of instance, 0-indexed
- **Name**: Name of the generated survey participant.
- **Gender**: Gender of the participant, either "Male" or "Female".
- **Age**: How old the participant is, ranging from 18 to 60.
- **City**: Which Indian city the participant came from.
- **Working Professional or Student**: Whether the participant is a "Working Professional" or a "Student".
- **Profession**: Profession of the participant who is a Working Professional. Value is Null wherever a participant is a Student.
- **Academic Pressure**: The participant's rating of their level of academic pressure, ranging from 1 to 5, with higher value meaning more pressure. Value is Null wherever a participant is a Working Professional.
- **Work Pressure**: The participant's rating of their level of work pressure, ranging from 1 to 5, with higher value meaning more pressure. Value is Null wherever a participant is a Student.
- **CGPA**: The participant's Cumulative Grade Point Average, ranging from 5 to 10. Value is Null wherever a participant is a Working Professional.
- **Study Satisfaction**: The participant's rating of their level of study satisfaction, ranging from 1 to 5, with higher value meaning more satisfaction. Value is Null wherever a participant is a Working Professional.

-
- **Job Satisfaction:** The participant's rating of their level of job satisfaction, ranging from 1 to 5, with higher value meaning more satisfaction. Value is Null wherever a participant is a Student.
 - **Sleep Duration:** How much sleep the participant gets everyday. Despite the name, this is a categorical feature.
 - **Dietary Habits:** How the participant thinks about their own dietary habits.
 - **Degree:** The participant's academic degree tag.
 - **Have you ever had suicidal thoughts ?:** Whether the participant had had suicidal thoughts throughout their life, either "Yes" or "No".
 - **Work/Study Hours:** How much time everyday, does the participant spend working/studying, ranging from 0 to 12.
 - **Financial Stress:** The participant's rating of their level of financial stress, ranging from 1 to 5, with higher values meaning more stress.
 - **Family History of Mental Illness:** Whether the participant's family had had any history with mental illness, either "Yes" or "No".
 - **Depression:** Diagnosis of whether the participant has depression, either 0 or 1. This should be the label that we will try to predict.

5.2. Key Observations and Insights

Structurally Missing Data

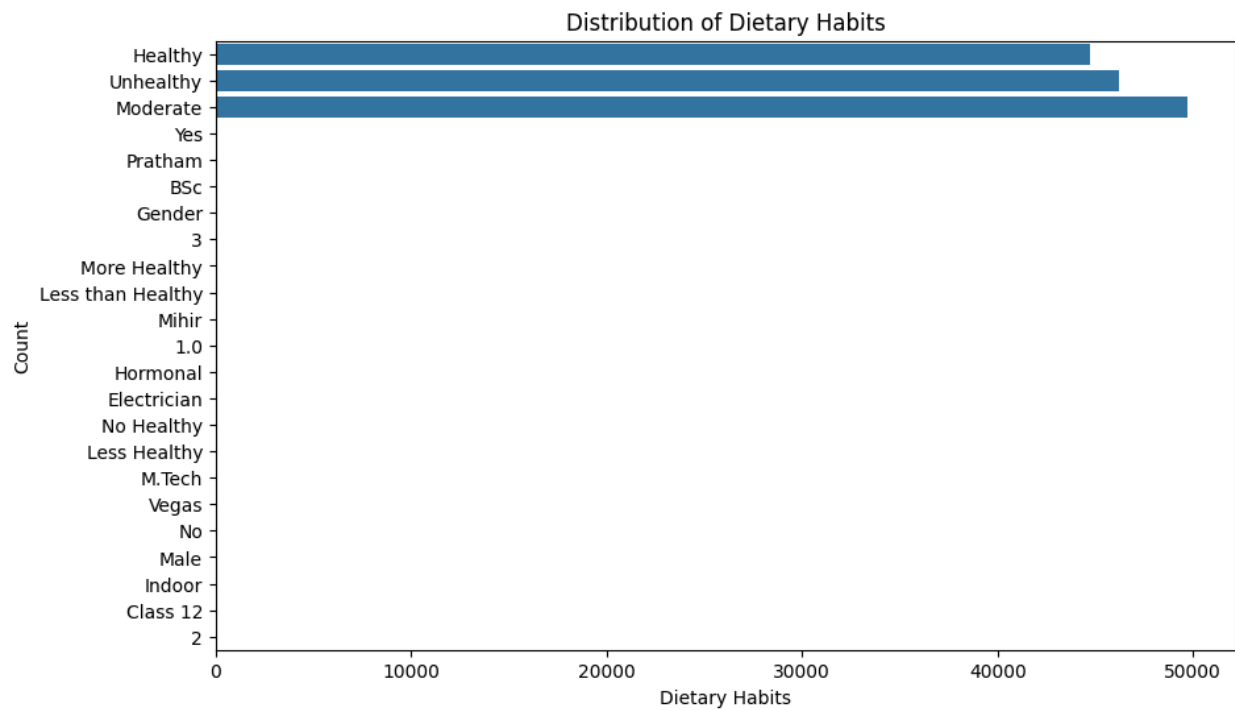
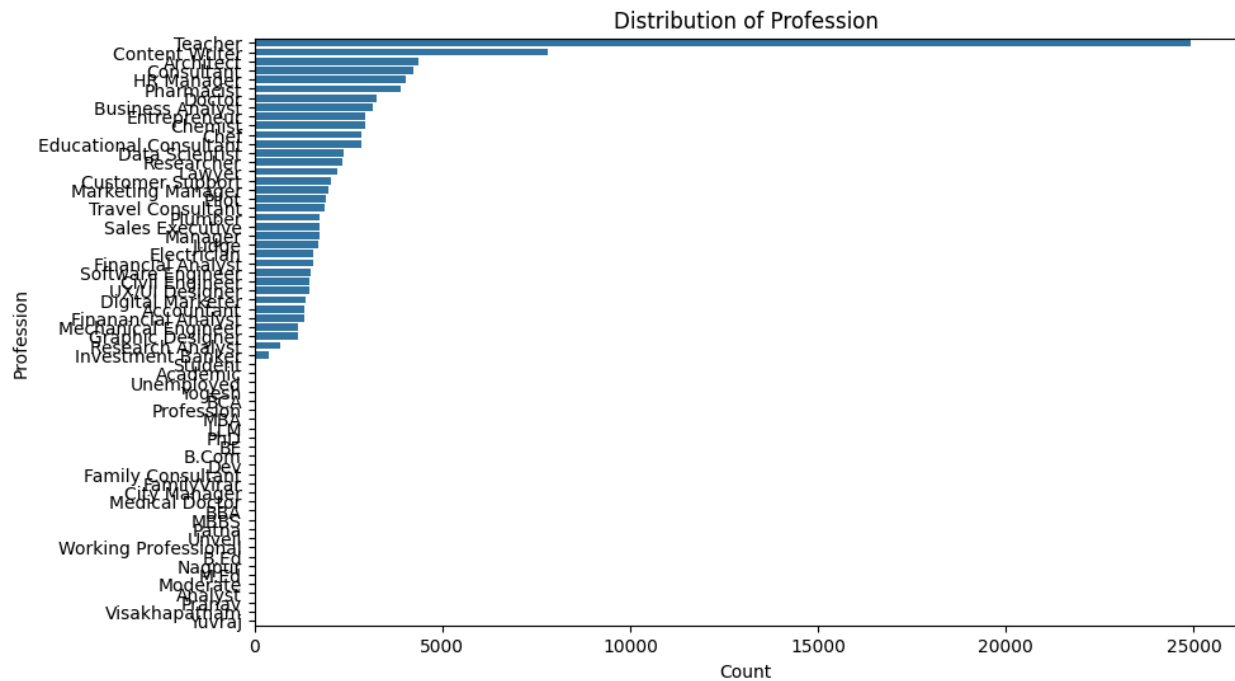
Some features depend on whether **Working Professional or Student** is a specific value in order to have a value, and will be Null otherwise. These include:

- **Profession, Work Pressure, Job Satisfaction:** Dependent on **Working Professional or Student** being "Working Professional".
- **Academic Pressure, CGPA, Study Satisfaction:** Dependent on **Working Professional or Student** being "Student".

Imputing these missing values should be the most simple, since all we need to do is add a value or label that is unique for when the dependency is not met, such as 0 for **Work Pressure** if **Working Professional or Student** is "Student".

Noisy Nominal Features

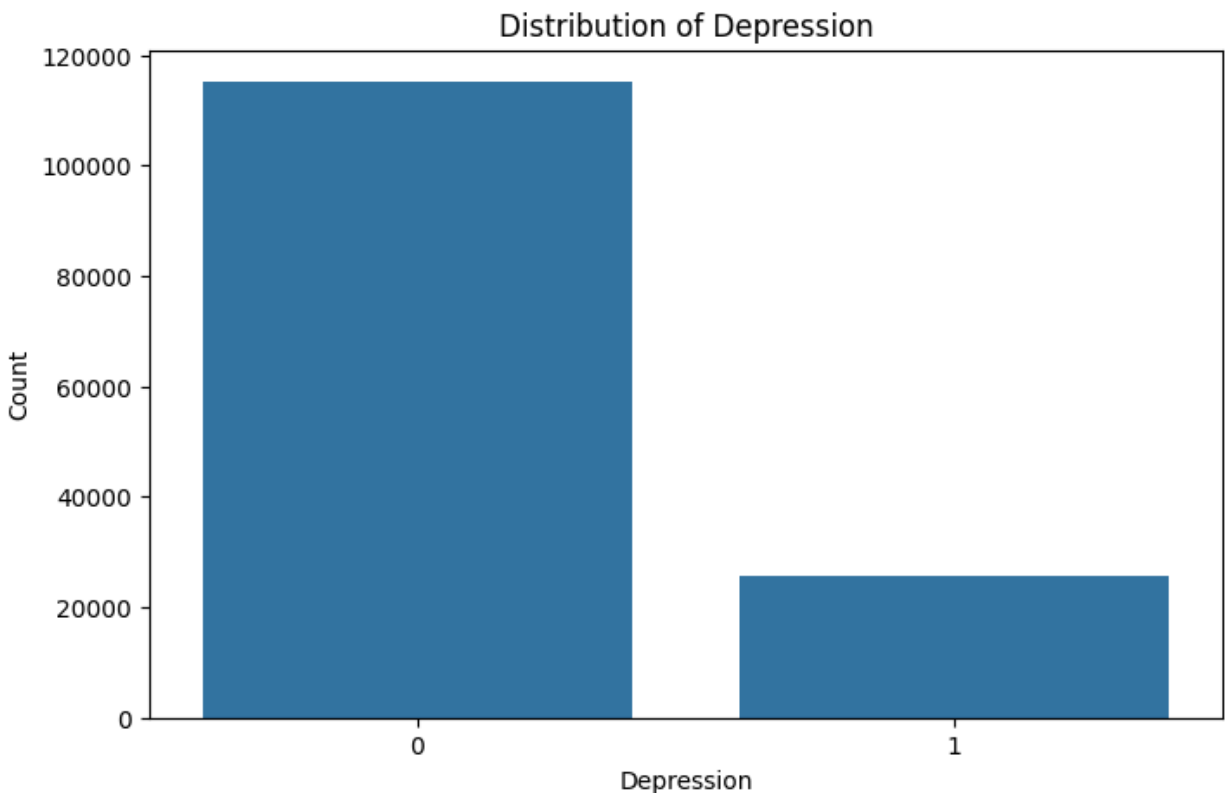
Nominal features like **Name**, **City**, **Profession** and **Degree** and even categorical features like **Sleep Duration** and **Dietary Habits** are super noisy.



Noises are mostly either values from another feature or possibly valid data but incorrectly formatted, and sometimes complete nonsense. This can be due to the original dataset being unclean when fed to the generator model, but it is most likely caused by the model itself picking up incorrect patterns and thus spewing out nonsensical data.

Imbalance Class Distribution

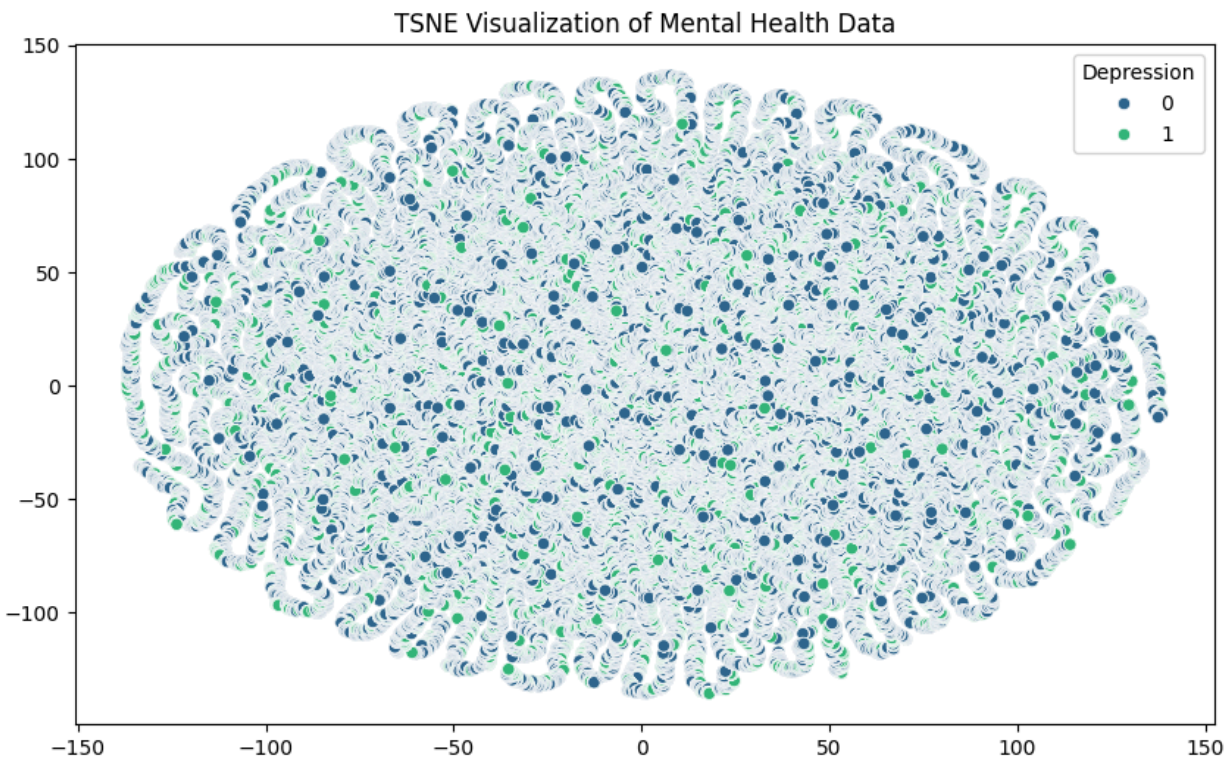
As can be seen from the distribution of class labels



The “0” class seems to outnumber the “1” class roughly 4 to 1. This should be taken into consideration when training prediction models.

Distribution Visualization

After trying to visualize the data distribution using TSNE, which is mainly useful for checking whether the classes form separable clusters in a low-dimensional projection. We have this abomination of a graph



The key takeaways here are straightforward:

- **Key observations:** Both classes (0 vs. 1) are scattered across the entire space with no clear, isolated clusters. Points of different labels overlap heavily, and no visible boundary or region dominated by a single class can be observed. Overall, the shape is a dense “blob” (the squiggles on the outline make it look like a brain tho), which often means TSNE couldn’t uncover any strong structure.
- **Insights:** The underlying features likely don’t contain simple, low-dimensional separable patterns, or patterns are more dependent on categorical features for forming decision boundaries. Linear or distance-based decision boundaries will not work well without at least some feature engineering. Any predictive signal is probably subtle, high-dimensional, or dependent on complex interactions rather than clear clusters.
- From what I know, this is pretty typical for synthetic or survey-style mental-health datasets: labels rarely form clean clusters.

6. Data Preprocessing

Preprocessing was performed on both the train and test datasets simultaneously. For more detailed visualization, please refer to the *data_preprocessing.ipynb* file.

The pipeline is heavily inspired by a Kaggle notebook from Adem Yilmaz on Kaggle.

6.1. Categorical features with only 2 values

All categorical features with 2 unique values, including **Gender**, **Working Professional or Student**, **Have you ever had suicidal thoughts ?**, and **Family History of Mental Illness** are all converted into binary features with this dictionary

```
1 # Binary mapping
2 gender_map = {'Male': 1,
3               'Female': 0}
4 suicidal_map = {'Yes': 1,
5                 'No': 0}
6 working_map = {'Working Professional': 1,
7                'Student': 0}
8 family_history_map = {'Yes': 1,
9                       'No': 0}
10
```

6.2. Noisy categorical and nominal features

Three nominal columns, **Name**, **City** and **Profession** are untouched in this step. Our reasoning for this decision is that since this dataset is synthetic, these features may carry patterns intentionally included by the generator. Dropping them risks discarding meaningful signals. Additionally, in case the nominal features don't carry any significant patterns, tree-based models like RandomForest and XGBoost, can just drop them into small, low-impact splits, enabling the models to absorb high-cardinality categorical features without severely overfitting.

Noise within the other categorical columns, however, all need to be processed manually. Columns **Sleep Duration**, **Dietary Habits**, and **Degree** are all “denoised” using mappings that map possibly valid but incorrectly formatted values into their correct counterparts. **Degree**, in particular, has an exceptionally large margin of distinct values needing standardization.

```

1 # Mapping for Degree
2 degree = {
3     "BCom": "B.Com", "B.Com": "B.Com", "B.Comm": "B.Com", "B BCom": "B.Com", "B B.Com": "B.Com",
4     "B.Tech": "B.Tech", "BTech": "B.Tech", "B.T": "B.Tech",
5     "BSc": "B.Sc", "B.Sc": "B.Sc", "Bachelor of Science": "B.Sc",
6     "BArch": "B.Arch", "B.Arch": "B.Arch",
7     "BA": "B.A", "B.A": "B.A",
8     "BBA": "BBA", "BB": "BBA", "B BA": "BBA", "B.B.Arch": "BBA",
9     "BCA": "BCA",
10    "BE": "BE",
11    "BEd": "B.Ed", "B.Ed": "B.Ed",
12    "BPharm": "B.Pharm", "B.Pharm": "B.Pharm",
13    "BHM": "BHM",
14    "LLB": "LLB", "LL B": "LLB", "LL BA": "LLB", "LL.Com": "LLB", "LLCom": "LLB",
15    "MCom": "M.Com", "M.Com": "M.Com",
16    "M.Tech": "M.Tech", "MTech": "M.Tech", "M.T": "M.Tech", "M_Tech": "M.Tech",
17    "MSc": "M.Sc", "M.Sc": "M.Sc", "Master of Science": "M.Sc",
18    "MBA": "MBA",
19    "MCA": "MCA",
20    "MD": "MD",
21    "ME": "ME",
22    "MEd": "M.Ed", "M.Ed": "M.Ed",
23    "MArch": "M.Arch", "M.Arch": "M.Arch",
24    "MPharm": "M.Pharm", "M.Pharm": "M.Pharm",
25    "MA": "MA", "M.A": "MA",
26    "MPA": "MPA",
27    "LLM": "LLM",
28    "PhD": "PhD",
29    "MBBS": "MBBS",
30    "CA": "CA",
31    "Class 12": "Class 12", "12th": "Class 12",
32    "Class 11": "Class 11", "11th": "Class 11"
33 }

```

This mapping will inevitably lead to missing data, as all values not declared in the mappings will be set to Null. They will all be imputed later.

6.3. Filling Structurally missing values

As stated in the Data Exploration section, some features have missing values whenever their dependencies are not met. Features related to stress and satisfaction levels are all filled with 0

```

# Fill Academic Pressure with 0 where Working Professional or Student is '1'
train.loc[(train['Working Professional or Student'] == 1) & (train['Academic Pressure'].isnull()), 'Academic Pressure'] = 0
test.loc[(test['Working Professional or Student'] == 1) & (test['Academic Pressure'].isnull()), 'Academic Pressure'] = 0

# and vice versa for Work Pressure
train.loc[(train['Working Professional or Student'] == 0) & (train['Work Pressure'].isnull()), 'Work Pressure'] = 0
test.loc[(test['Working Professional or Student'] == 0) & (test['Work Pressure'].isnull()), 'Work Pressure'] = 0

```

while the **Profession** feature is filled with a new category "Not Applicable".

```
# Fill in Profession with 'Not Applicable' where Working Professional or Student is '0'
train.loc[(train['Working Professional or Student'] == 0) & (train['Profession'].isnull()), 'Profession'] = 'Not Applicable'
test.loc[(test['Working Professional or Student'] == 0) & (test['Profession'].isnull()), 'Profession'] = 'Not Applicable'
```

CGPAs for all Working Professionals are also filled with 0s.

```
# Fill in CGPA with 0 where Working Professional or Student is '1'
train.loc[(train['Working Professional or Student'] == 1) & (train['CGPA'].isnull()), 'CGPA'] = 0
test.loc[(test['Working Professional or Student'] == 1) & (test['CGPA'].isnull()), 'CGPA'] = 0
```

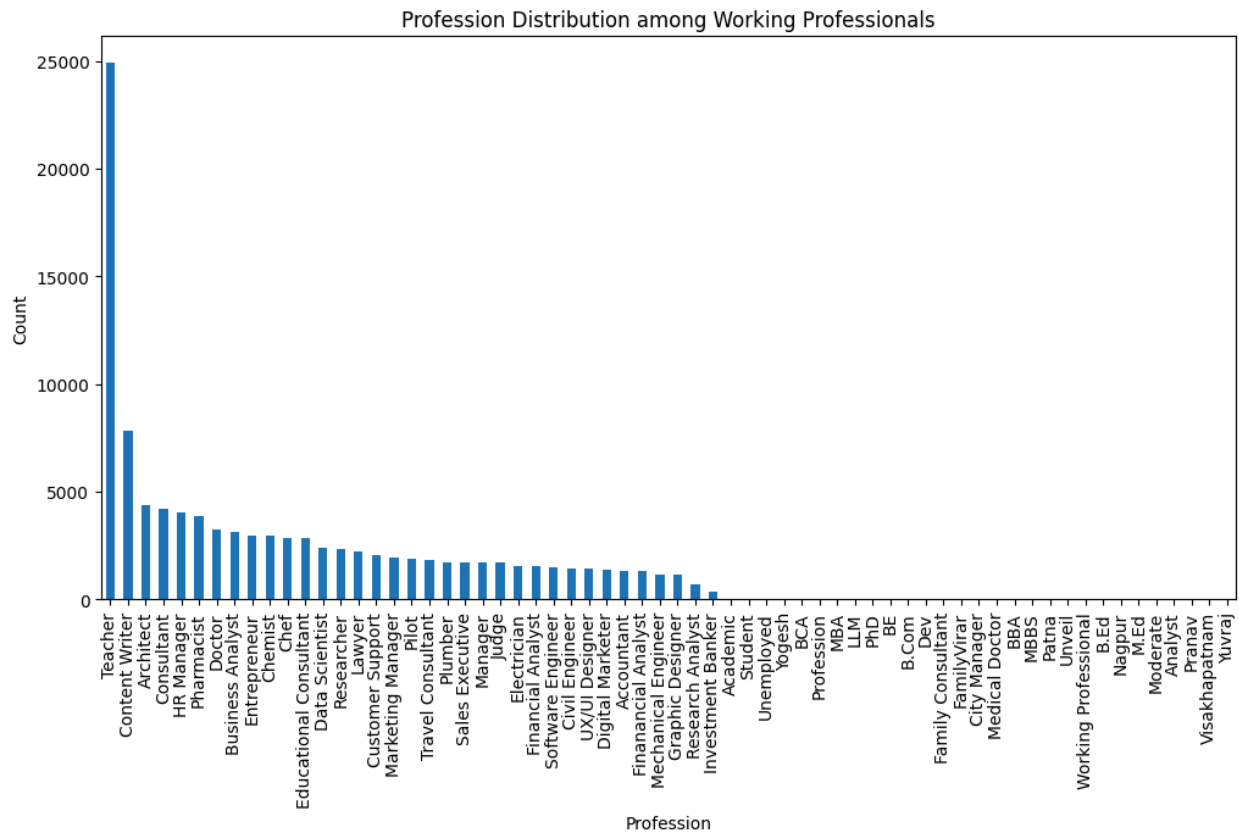
Of course, actual missing values do exist within these columns, so they will also be imputed.

6.4. Imputing Actual Missing Values

Overall, the training data is sufficiently filled, with **Profession** and **Degree** having the highest number of missing values, though they are still less than 10% of the whole thing. The same thing can be said about the test data.

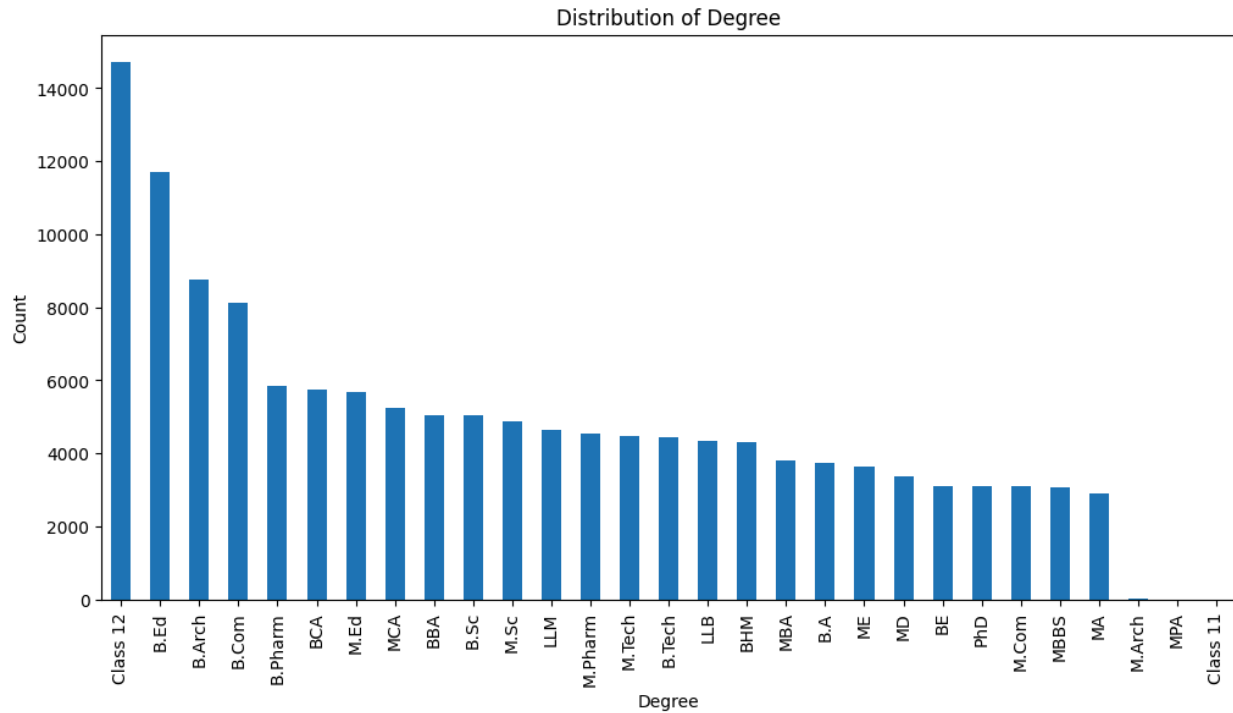
Missing values in training set:		Missing values in test set:	
id	0	id	0
Name	0	Name	0
Gender	0	Gender	0
Age	0	Age	0
City	0	City	0
Working Professional or Student	0	Working Professional or Student	0
Profession	8763	Profession	5886
Academic Pressure	9	Academic Pressure	7
Work Pressure	20	Work Pressure	10
CGPA	9	CGPA	9
Study Satisfaction	10	Study Satisfaction	8
Job Satisfaction	17	Job Satisfaction	9
Sleep Duration	20	Sleep Duration	13
Dietary Habits	22	Dietary Habits	27
Degree	3375	Degree	2211
Have you ever had suicidal thoughts ?	0	Have you ever had suicidal thoughts ?	0
Work/Study Hours	0	Work/Study Hours	0
Financial Stress	4	Financial Stress	0
Family History of Mental Illness	0	Family History of Mental Illness	0
Depression	0		
dtype: int64		dtype: int64	

For the **Profession** column, since “Teacher” absolutely dominated its distribution



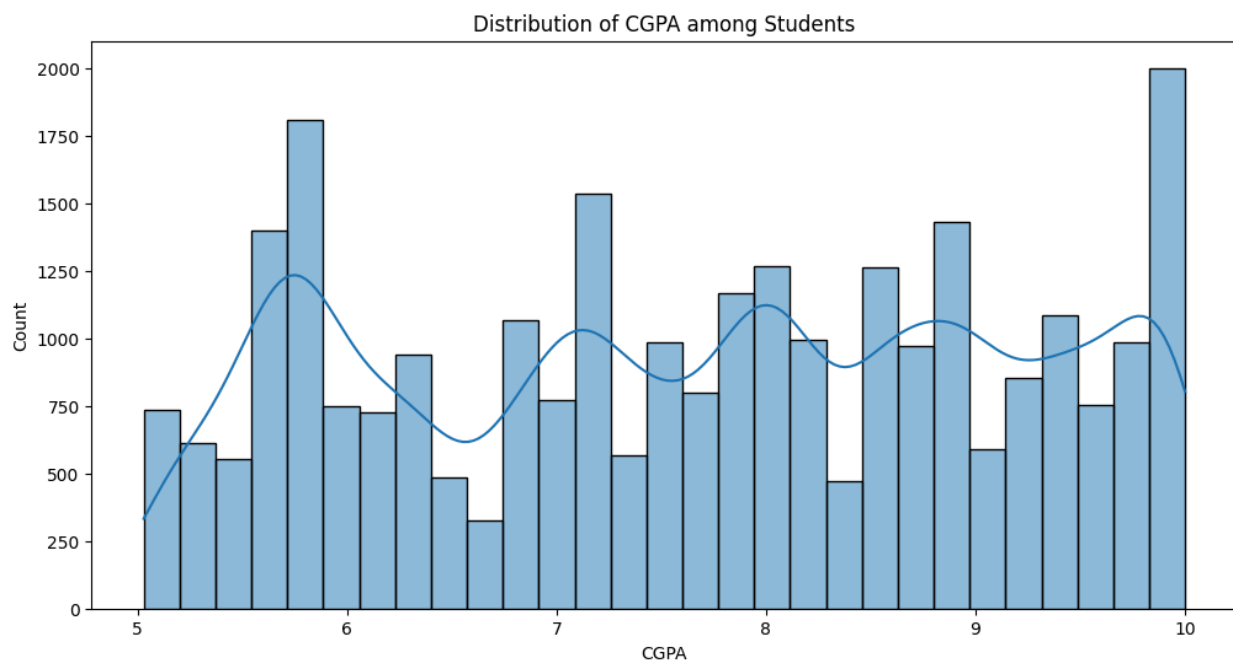
It's probably best to use mode imputation for its missing values.

Next is **Degree**, whose distribution goes as follow



It's not quite the same as that of **Profession**. Here, the frequencies of “minority” values are much more prominent. The imputation method used here is KNN Imputation.

The same method (KNN Imputation) is also utilized to fill in missing data of the feature **CGPA**, but for a different reason. The distribution of CGPA values goes like this



Which looks nothing like the usual standard distribution often seen in scoring. This makes the usual mean, median or mode imputation likely to skew the distribution of values, ruining the pattern intentionally created by the generator model. KNN imputation preserves the relationships between features, which can lead to better model performance compared to simpler imputation methods like mean or median imputation. (*Handling Missing Data With KNN Imputer*, 2025)

Missing values of other features are imputed as follow

Feature	Imputation method
Academic Pressure	mode
Work Pressure	mean
Study Satisfaction	mode
Job Satisfaction	mode
Sleep Duration	weighted random sampling
Dietary Habits	mode
Financial Stress	median

6.5. Processing Duplicated Rows

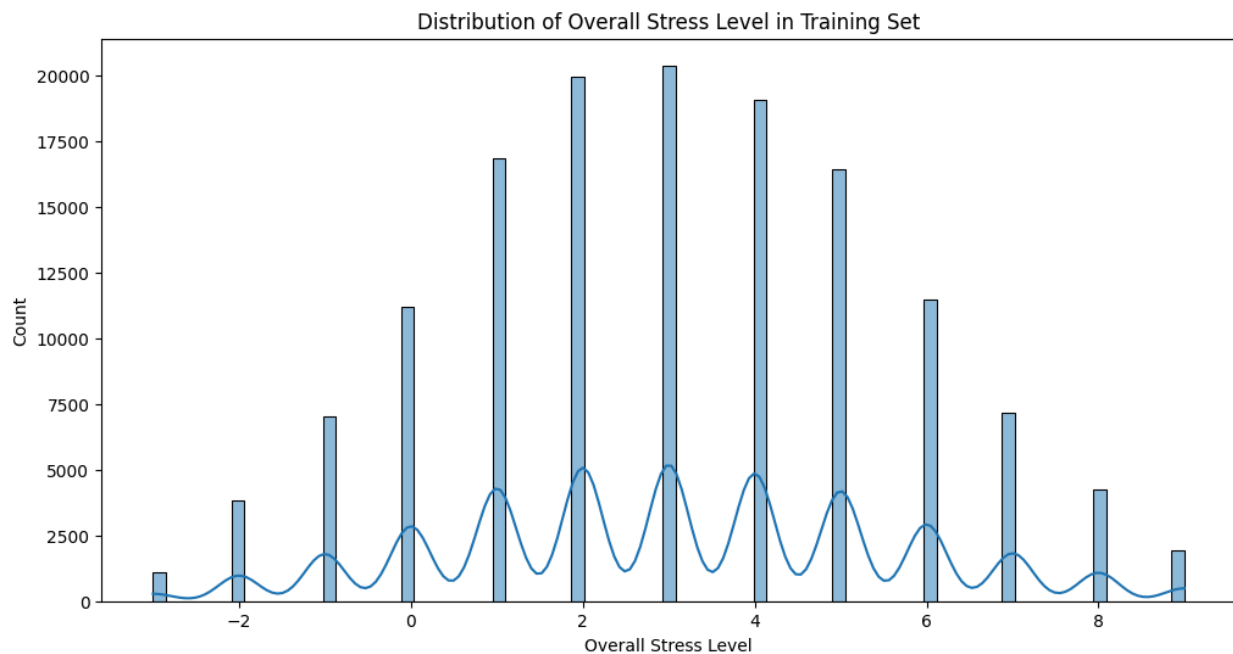
Fortunately, the dataset does not have any duplicated rows. No further action is needed.

7. Feature Engineering

Except for the **id** column, all features after preprocessing are used for training models. As for manufacturing new features, one new feature is created. **Overall Stress Level**, the newly engineered feature, aims to provide a more comprehensive summary of each participant's stress level by combining related factors. Assuming stress from different sources is additive, and satisfaction from what we do can help relieve stress, below is the formula to calculate the new feature:

$$\begin{aligned} \text{Overall Stress Level} = & \text{Academic Pressure} + \text{Work Pressure} + \text{Financial Stress} \\ & - \text{Study Satisfaction} - \text{Job Satisfaction} \end{aligned}$$

This is also one of the reasons the pressure and satisfaction values are set to 0 instead of any other value. After the preprocessing steps, Students will all have **Work Pressure** and **Job Satisfaction** set to 0, while Working Professionals will have **Academic Pressure** and **Study Satisfaction** set to 0. This ensures that **Overall Stress Level** accurately reflects the stressors relevant to each group. Interestingly enough, distribution of this new feature is a near perfect normal distribution



8. Model Development

Throughout this project, a total of 3 distinct model architectures were developed

8.1. Tree-based Ensemble Model

- Script: *xgboost_training.ipynb*
- Library used: xgboost
- Architecture: gradient boosted trees (*XGBoost Documentation*, n.d.)
- Parameters:
 - `learning_rate`: Learning rate of the gradient, same as any other method that uses gradient descent.
 - `max_depth`: Maximum allowed depth of each tree in the ensemble model, same as decision tree and any other tree-based model.

-
- `n_estimators`: Maximum number of estimators (trees) to add into the ensemble model, since one is added every boosting round (iteration), this value is the same as epoch in models that use gradient descent.
 - `reg_lambda`: L2 regularization scale
 - `early_stopping_rounds`: Number of boosting rounds without improvement before the algorithm stops early.
 - `min_child_weight`: Minimum sum of instance weight needed in a child
 - `colsample_bytree`: Subsample ratio of columns when constructing each tree
 - `Subsample`: Subsample ratio of the training instances

8.2. Stacking Ensemble Model

- Script: *stacking_ensemble_training.ipynb*
- Library used: `mlxtend` (Raschka, n.d.), `scikit-learn` (scikit-learn, n.d.)
- Architecture:
 - Base models:
 - **LogisticRegression**: Provides a strong linear baseline that performs well on high-dimensional data and offers stable, interpretable decision boundaries.
 - **SGDClassifier**: Adds a fast, regularized linear model that can capture patterns missed by closed-form logistic regression, especially on large or sparse feature spaces.
 - **PassiveAggressiveClassifier**: Contributes an aggressive margin-based linear learner that reacts strongly to misclassified points, improving diversity within the linear family.
 - **DecisionTreeClassifier**: Supplies a non-linear model that captures hierarchical feature interactions and complements the linear models' limitations.
 - **LinearSVC**: Adds a robust max-margin classifier that often generalizes well on noisy datasets and strengthens the ensemble's linear decision boundaries.
 - **GaussianNB**: Provides a probabilistic model that excels when features are conditionally independent, contributing a very different bias to the ensemble.

-
- **ExtraTreesClassifier**: Contributes a high-variance, randomized tree ensemble that captures complex non-linear relationships with strong feature-interaction diversity.
 - **AdaBoostClassifier**: Adds another boosting-based learner that focuses on hard-to-classify samples, improving the ensemble's sensitivity to subtle patterns.
 - Meta-model: **LogisticRegression**: Serves as a stable, low-variance combiner that optimally weighs the heterogeneous predictions from all base learners.
 - Parameters: Each base model has their own set of parameters to work with.

8.3. MLP (Multilayer Perceptron) Model

- Script: *mlp_training.ipynb*
- Library used: scikit-learn, specifically `sklearn.neural_network.MLPClassifier` (*MLPClassifier — Scikit-Learn 1.7.2 Documentation*, n.d.)
- Architecture:
 - Input Layer: Automatically sized to match the number of features in your data
 - Hidden Layers: Configurable number of layers and neurons
 - Output Layer: Automatically sized based on the number of classes
- Parameters:
 - `activation`: Activation functions used by the neurons
 - `alpha`: L2 regularization scale
 - `batch_size`: size of batches the training data is split into
 - `early_stopping`: whether to stop the algorithm if many epochs has elapsed without meaningful improvement
 - `hidden_layer_sizes`: Sizes of the hidden layers
 - `learning_rate_init`: Initial value of learning rate.
 - `max_iter`: Maximum number of iterations to go through.

9. Experiments and Results

Model evaluation is performed using K-Fold cross validation. By splitting the dataset into 10 folds, then using 90% as train and 10% as validation, we can get a more generalized knowledge of the model's performance on this particular dataset.

9.1. Hyperparameter Tuning

Out of the 3 models, the XGBoost model is the only one tuned manually. The stacking ensemble model and the mlp model are tuned using gridsearch. This method takes an obscene amount of time, considering the stacking ensemble model has 8 base models, and the mlp model has quite a few tunable hyperparameters, each having multiple possible values. Out of the 3 models:

- The XGBoost model's manual tuning took the least time. Even at 500 iterations, the whole pipeline takes less than 20 minutes to generate predictions for the test data.
- Each of the 8 base models of the stacking ensemble model is tuned separately (but still on the same notebook) using **GridSearchCV**, a Python library optimized for grid search using K-Fold cross-validation. The whole process of tuning, training the final model on the whole dataset, and generating predictions for the test data take less than an hour to complete.
- The single mlp model requires the most time to tune. Clocking in at 9 and a half hours, the tuning has to be performed on a [Kaggle notebook](#) instead of locally on a laptop like the other two. Then the final model using the recommended hyperparameters is trained locally.

9.2. Evaluation Results

Tree-based Ensemble Model

	precision	recall	f1-score	support
0	0.9730	0.9369	0.9545	11513
1	0.7577	0.8826	0.8144	2556
macro avg	0.8653	0.9098	0.8844	14070
weighted avg	0.9339	0.9269	0.9290	14070
Average Accuracy: 0.9269				

Most likely due to being unoptimized, the XGBoost model performs the worst out of the three models in terms of overall accuracy. As of the latest version, it suffers from poor precision when predicting True labels, which heavily affects its overall performance.

Stacking Ensemble Model

	precision	recall	f1-score	support
0	0.9662	0.9543	0.9602	11513
1	0.8049	0.8495	0.8266	2556
macro avg	0.8855	0.9019	0.8934	14070
weighted avg	0.9369	0.9352	0.9359	14070
Average Accuracy: 0.9352				

Boasting a much higher f1-score on both classes, this model also has the second highest overall accuracy.

MLP Model

	precision	recall	f1-score	support
0	0.9587	0.9664	0.9625	11513
1	0.8432	0.8125	0.8275	2556
macro avg	0.9010	0.8894	0.8950	14070
weighted avg	0.9377	0.9385	0.9380	14070
Average Accuracy: 0.9385				

The best model so far in terms of overall accuracy. The f1-score of this model also barely has an edge over that of the stacking ensemble model.

9.3. Test Results

As of their latest versions

 HCMUS-22KHDL-Cloud_Djata_xgboost_v2.csv Complete (after deadline) · 7h ago	0.92465	0.92558
 HCMUS-22KHDL-Cloud_Djata_stack_v5.csv Complete (after deadline) · 38s to go	0.93544	0.93624

 HCMUS-22KHDL-Cloud_Djata_MLP_v2.csv Complete (after deadline) · 7h ago	0.93812	0.93965
--	----------------	----------------

The predictions from the mlp model achieved the best results, both in Private Score (left) and Public Score (right). However, this version of the model is actually not the best. The prediction that achieved the highest scores is...

 HCMUS-22KHDL-Cloud_Djata_MLP_v1.csv Complete (after deadline) · 6d ago	0.93868	0.93997
--	----------------	----------------

...the first version of the mlp model. “What is the difference between the two versions?”, you may ask. The first model was actually trained with the **Name** and **City** columns included in the dataset, whereas the second version was not. So by analyzing the subtle patterns and noises introduced by the generator model from the Kaggle competition, the prediction quality actually increases.

10. Conclusion

The project developed an end-to-end pipeline to predict the binary depression label from a synthetic survey dataset. Robust preprocessing, including manual cleaning, several imputation strategies and a composite stress feature produced stable inputs. Three modeling strategies were developed, varying in architectural complexity. All models performed relatively well, with all scores in the Kaggle competition being above 0.92, and one model almost reaching 0.94 (though only in public score). Experiments also showed that overfitting on noisy data may have certain beneficial effects on predictions of non-natural, synthetic data.

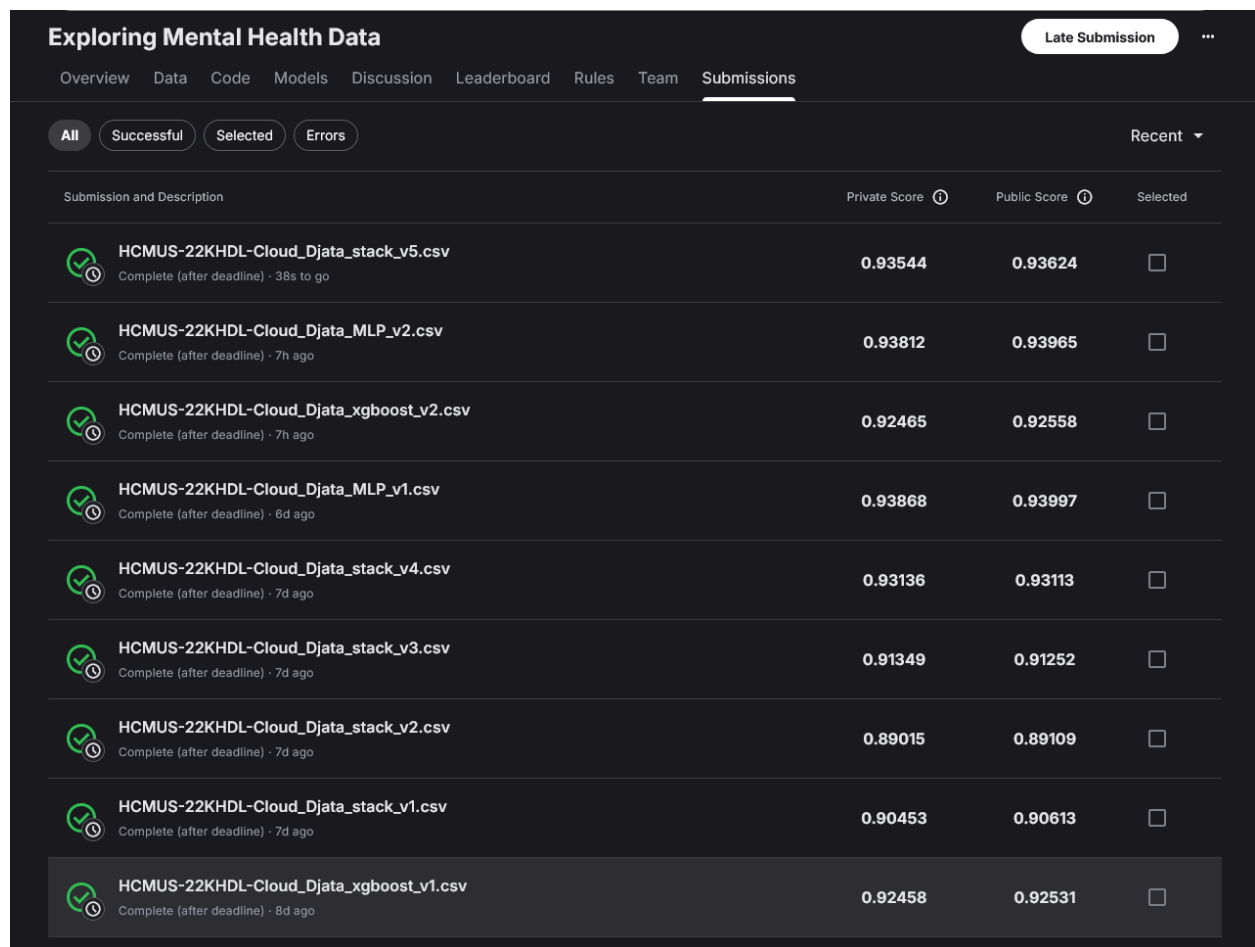
11. Future Work and Improvements

- More systematic hyperparameter search and compute-aware tuning (e.g., Bayesian optimization, larger Grid/Random search) for XGBoost and MLP to squeeze out additional gains.
- Better handling of class imbalance beyond `scale_pos_weight`: try stuff like resampling (SMOTE/ADASYN), ensemble reweighting, threshold tuning and probability calibration.
- Feature refinement: evaluate importance and perform automated feature selection, construct interaction features and test alternative encodings (one-hot or

embeddings instead of juggling between label and target encoding) for high-cardinality categorical fields.

- Model explainability and robustness: compute SHAP/Permutation importances, uncertainty estimates, and test model stability under distribution shifts.
- Validation and ensembling: adopt nested CV for more reliable model selection, experiment with weighted/blended ensembles and stacking variants, and verify final ensemble on hold-out data or external datasets.
- Deployment/readiness: add reproducible pipelines (serialization of preprocessors and models) and lightweight inference notebooks or scripts for submission generation only.

12. Screenshot Evidence of All Submissions



Exploring Mental Health Data				Late Submission	...
Overview Data Code Models Discussion Leaderboard Rules Team Submissions					
All Successful Selected Errors		Recent ▾			
Submission and Description		Private Score ⓘ	Public Score ⓘ	Selected	
	HCMUS-22KHDL-Cloud_Djata_stack_v5.csv Complete (after deadline) · 38s to go	0.93544	0.93624	☐	
	HCMUS-22KHDL-Cloud_Djata_MLP_v2.csv Complete (after deadline) · 7h ago	0.93812	0.93965	☐	
	HCMUS-22KHDL-Cloud_Djata_xgboost_v2.csv Complete (after deadline) · 7h ago	0.92465	0.92558	☐	
	HCMUS-22KHDL-Cloud_Djata_MLP_v1.csv Complete (after deadline) · 6d ago	0.93868	0.93997	☐	
	HCMUS-22KHDL-Cloud_Djata_stack_v4.csv Complete (after deadline) · 7d ago	0.93136	0.93113	☐	
	HCMUS-22KHDL-Cloud_Djata_stack_v3.csv Complete (after deadline) · 7d ago	0.91349	0.91252	☐	
	HCMUS-22KHDL-Cloud_Djata_stack_v2.csv Complete (after deadline) · 7d ago	0.89015	0.89109	☐	
	HCMUS-22KHDL-Cloud_Djata_stack_v1.csv Complete (after deadline) · 7d ago	0.90453	0.90613	☐	
	HCMUS-22KHDL-Cloud_Djata_xgboost_v1.csv Complete (after deadline) · 8d ago	0.92458	0.92531	☐	

13. References

Handling Missing Data with KNN Imputer. (2025, July 23). GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/handling-missing-data-with-knn-imputer/>

Kaggle. (n.d.). *Playground Series - Exploring Mental Health Data*. Kaggle.

<https://www.kaggle.com/competitions/playground-series-s4e11/overview>

MLPClassifier — *scikit-learn 1.7.2 documentation*. (n.d.). Scikit-learn.

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Raschka, S. r. (n.d.). *Welcome to mlxtend's documentation!* github.io.

<https://rasbt.github.io/mlxtend/>

scikit-learn. (n.d.). *Scikit-learn*. scikit-learn: machine learning in Python — scikit-learn 1.7.2 documentation. <https://scikit-learn.org/stable/index.html>

Sharma, S. (n.d.). *Depression Survey/Dataset for Analysis*. Kaggle.

<https://www.kaggle.com/datasets/sumansharmadataworld/depression-surveydataset-for-analysis>

XGBoost Documentation. (n.d.). XGBoost Documentation — xgboost 3.1.1 documentation.

<https://xgboost.readthedocs.io/en/stable/>

Yilmaz, A. (n.d.). *This code fixed my depression*. Kaggle.

<https://www.kaggle.com/code/adyiemaz/this-code-fixed-my-depression>